Text Splitting — Common Mistakes, Limitations & Best Practices

1. Common Mistakes

Mistake 1 — Thinking Chunk Size Means Tokens

Wrong.

When the default:

length_function=len

is used, chunk size is measured using:

Python String Length

not model tokens.

Simple memory:

chunk_size

↓

Measured by length_function

With:

length_function=len

it means:

Character Length

not:

Token Count

Mistake 2 — Thinking All Chunks Must Have Equal Length

Wrong.

Suppose:

chunk_size = 5

It does not mean every chunk must contain exactly 5 characters.

Example:

Chunk 1: ABCDE

Chunk 2: FGHIJ

Chunk 3: KL

The final chunk is shorter.

So:

chunk_size

↓

Target Maximum

not:

Exact Required Length

Mistake 3 — Thinking Chunk Overlap Must Always Be Zero

Wrong.

Controlled overlap may help preserve related information across chunk boundaries.

Example:

Chunk 1: ABCDE

Chunk 2: DEFGH

Repeated text:

DE

helps maintain continuity.

Mistake 4 — Thinking More Overlap Is Always Better

Wrong.

Too much overlap can create:

- Duplicate content
- More chunks
- More storage
- More processing
- More unnecessary repetition

So:

No Overlap

↓

May Lose Continuity

Controlled Overlap

↓

Useful

Too Much Overlap

↓

Too Much Duplication

Mistake 5 — Thinking Separators and Chunk Size Are the Same Wrong.

They have different responsibilities.

Separator

↓

WHERE text may be divided

Chunk Size

↓

HOW LARGE the final chunk should be

Easy memory:

Separator → WHERE?

Chunk Size → HOW MUCH?

Mistake 6 — Thinking Text Splitting Is the Same as Embedding

Wrong.

Text splitting and embedding are two different steps.

Text Splitting

Large Text

↓

Smaller Text Chunks

Embedding

Text

↓

Numerical Vector

So:

Text Splitting

↓

DIVIDE

Embedding

↓

CONVERT TO NUMBERS

Mistake 7 — Thinking Splitting Automatically Produces Better AI Answers

Wrong.

Text splitting is only a preparation step.

Good AI answers may also depend on:

- Good chunk design
- Useful retrieval
- Relevant context
- Good embeddings
- Proper model behaviour
- Correct application logic

So:

Good Splitting

≠

Guaranteed Good Answer

Mistake 8 — Ignoring Metadata

When splitting LangChain Document objects, source information may be very important.

Example:

```
{
   "source": "company_policy.pdf",
   "page": 2
}
```

After splitting, we should ensure useful metadata remains available.

Why?

Because later we may want to know:

Which File?

Which Page?

Which Source?

Mistake 9 — Using One Chunk Size for Every Application

Different applications may require different chunking strategies.

For example:

Short FAQ

↓

Smaller Chunks May Work

Long Policies

↓

Larger Chunks May Be Useful

Code

↓

Different Splitting Strategy

So:

There is no single perfect chunk size for every application.

2. Best Practices

1. Understand the type of document before choosing a splitting strategy.
2. Treat chunk_size as a target maximum, not an exact length.
3. Use controlled overlap where continuity is important.
4. Avoid unnecessarily large overlap.
5. Choose meaningful separators where possible.
6. Preserve important metadata.
7. Do not confuse text splitting with embedding.

8. Test different chunk sizes for different applications.
9. Evaluate retrieval quality, not only chunk count.
10. Remember that splitting is only one step in the RAG pipeline.

3. Master Formula

Large Document

↓

Text Splitting

↓

Useful Chunks

↓

Embedding

↓

Vector Storage

↓

Retrieval

↓

LLM

↓

Answer

Important:

Text Splitting

↓

Preparation Step

It does not itself produce the final answer.

4. MCQ 1

Why do we split large documents in LangChain?

A. To delete unnecessary files
B. To divide large text into smaller useful chunks
C. To train the LLM automatically
D. To convert text directly into images

Correct Answer: B — To divide large text into smaller useful chunks

Explanation

Text splitting prepares smaller pieces that can be processed and retrieved more efficiently.

Learning Conclusion

“Large knowledge becomes useful when it is divided into meaningful pieces.”

5. MCQ 2

What is a chunk?

A. A complete AI model
B. A type of API key
C. A smaller piece of text
D. A vector database

Correct Answer: C — A smaller piece of text

Explanation

A chunk is a portion of the original document created during text splitting.

Learning Conclusion

“A chunk is simply a small piece taken from a larger knowledge source.”

6. MCQ 3

If chunk_size=5, must every output chunk contain exactly 5 characters?

A. Yes, always
B. Yes, except for PDF documents
C. Only when using an LLM
D. No, actual lengths depend on the splitting method

Correct Answer: D — No, actual lengths depend on the splitting method

Explanation

Chunk size normally acts as a target maximum, so actual chunk lengths can vary.

Learning Conclusion

“Chunk size gives a limit, not a guarantee that every chunk will be equal.”

7. MCQ 4

What is chunk overlap?

A. Text repeated across neighbouring chunks
B. The total number of documents
C. The size of the original PDF
D. The total input tokens of an LLM

Correct Answer: A — Text repeated across neighbouring chunks

Explanation

Overlap repeats some nearby text to help preserve information across chunk boundaries.

Learning Conclusion

“A little repetition can protect important context from being broken.”

8. MCQ 5

What is one disadvantage of using too much overlap?

A. It permanently deletes the text
B. It creates unnecessary duplicate content
C. It disables the LLM
D. It removes all metadata

Correct Answer: B — It creates unnecessary duplicate content

Explanation

Higher overlap can increase the number and total volume of chunks.

Learning Conclusion

“Overlap should preserve context, not create unnecessary repetition.”

9. MCQ 6

Which separator commonly represents a blank line between paragraphs?

A. A space only
B. An empty dictionary
C. \n\n
D. The pipe operator

Correct Answer: C — \n\n

Explanation

Two newline characters commonly represent a blank line between paragraphs.

Learning Conclusion

“Paragraph boundaries are natural places to divide meaningful text.”

10. MCQ 7

With the default length_function=len, what is used to measure chunk size?

A. Python string length
B. Number of PDF pages
C. Number of AI models
D. Database row count

Correct Answer: A — Python string length

Explanation

The len() function measures the length of the Python string.

Learning Conclusion

“Chunk size depends on the chosen length function, not automatically on LLM tokens.”

11. MCQ 8

What does split_text() return?

A. One AIMessage
B. A list of strings
C. An embedding vector
D. A database connection

Correct Answer: B — A list of strings

Explanation

split_text() divides an input string and returns the resulting chunks as:

list[str]

Learning Conclusion

“Text goes in, smaller text strings come out.”

12. MCQ 9

What does split_documents() normally return?

A. Only one long string
B. A list of numbers
C. One API key per chunk
D. A list of Document objects

Correct Answer: D — A list of Document objects

Explanation

It creates smaller Document objects while preserving useful metadata from the original Document.

Learning Conclusion

“Document splitting keeps both the chunk content and its source identity.”

13. MCQ 10

Which statement correctly describes the role of text splitting in RAG?

A. Text splitting alone generates the final answer
B. Text splitting automatically trains an LLM
C. Text splitting prepares smaller pieces for later embedding and retrieval
D. Text splitting replaces document loaders

Correct Answer: C — Text splitting prepares smaller pieces for later embedding and retrieval

Explanation

Text splitting is one preparation step before embeddings, vector storage and retrieval.

Learning Conclusion

“Splitting prepares knowledge; retrieval finds it; the LLM uses it to answer.”

14. MCQ Answer Summary

MCQ	Correct Answer
1	B
2	C
3	D
4	A
5	B
6	C
7	A
8	B
9	D
10	C

15. Quick Revision

Chunk

↓

Small Piece of Text

Chunk Size

↓

Target Maximum Length

Chunk Overlap

↓

Repeated Text Between Chunks

Separator

↓

Possible Splitting Point

split_text()

↓

list[str]

split_documents()

↓

list[Document]

Text Splitting

↓

Preparation for Later

Embedding and Retrieval

16. Final RAG Memory Formula

Document Loader

↓

Document

↓

Text Splitter

↓

Chunks

↓

Embeddings

↓

Vector Database

↓

Retriever

↓

Relevant Chunks

↓

LLM

↓

Answer

17. Final Summary Conclusion

Text Splitting is not simply cutting text into pieces. It is about preparing useful pieces of information so that AI applications can process, store, search and retrieve knowledge effectively.